

Technical Debt and Performance Optimization Analysis

Full Stack Development Internship Report

Submitted to:

Prosensia

Submitted by:

Tayyaba Shakoor

Full Stack Development Intern

03 June 2026

1. Executive Summary

This report analyzes technical debt in frontend development, explains React's component lifecycle, and demonstrates how missing cleanup functions in `useEffect` cause memory leaks. It provides optimized code patterns to prevent browser performance degradation.

The concepts are illustrated using components built for this internship — including `Button`, `Card`, and `Modal` — developed with `TypeScript` and `Tailwind CSS`.

2. Technical Debt Analysis

2.1 Definition

Technical debt is the future cost of rework caused by choosing quick, easy solutions instead of better, long-term approaches. Like financial debt, if not paid back through refactoring, it accumulates interest in the form of slower development, more bugs, and harder maintenance.

2.2 Common Causes in Frontend Development

Cause	Impact	Interest Accumulation
No TypeScript interfaces	Runtime errors, poor autocomplete	Hours of debugging
Missing cleanup in <code>useEffect</code>	Memory leaks, slow performance	Browser crashes
Copy-pasting code	Duplicate logic, inconsistent behavior	Bug fixes in multiple places
Inline styles	No design system, hard to maintain	CSS inconsistencies everywhere
No component reusability	Large files, hard to test	Refactoring becomes a nightmare

2.3 How This Task Avoids Technical Debt

Practice	Implementation
TypeScript interfaces	Every component has interface <code>Props</code> with proper types
Reusable components	<code>Button</code> , <code>Card</code> , <code>Modal</code> can be used anywhere with different props

Practice	Implementation
Tailwind CSS	Consistent utility classes, zero custom CSS files
Atomic Design	Clear separation: atoms, molecules, organisms, templates, pages
Cleanup in useEffect	Modal component properly cleans up event listeners and body scroll

3. React Component Rendering Lifecycle

3.1 The Three Phases

Phase	What Happens	When It Occurs
Mounting	Component is created and inserted into DOM	First time component appears
Updating	Component re-renders due to state or prop changes	User interaction, data fetching, timer events
Unmounting	Component is removed from DOM	User navigates away, conditional rendering hides it

3.2 useEffect Hook Lifecycle

Example 1 — Runs once after mount (mounting phase):

```
useEffect(() => {  
  console.log("Component mounted");  
}, []);
```

Example 2 — Runs after mount and every time dependency changes:

```
useEffect(() => {  
  console.log("Dependency changed");  
}, [dependency]);
```

Example 3 — Runs after every single render:

```
useEffect(() => {  
  console.log("Rendered");  
});
```

Example 4 — With cleanup, runs on unmount:

```
useEffect(() => {  
  // Setup code  
  return () => {  
    // Cleanup code runs on unmount  
  };  
}, []);
```

4. The Problem: useEffect Without Cleanup

4.1 What is a Memory Leak?

A memory leak occurs when the browser cannot free memory that is no longer needed. The browser's memory usage grows over time, eventually causing:

- Slow page performance
- High CPU usage
- Browser tab crashes
- Entire browser freezing

4.2 BAD Code Example (Memory Leak)

```
function BadComponent() {  
  useEffect(() => {  
    const interval = setInterval(() => {  
      console.log("Still running...");  
    }, 1000);  
  
    // NO CLEANUP FUNCTION  
    // The interval will NEVER stop  
  }, []);  
  
  return <div>Bad Component</div>;  
}
```

4.3 What Happens Step by Step

Step	Result	Memory State
1	Component mounts, interval starts	Normal
2	Component unmounts but interval keeps running	Interval still active in background
3	User returns — NEW interval starts	2 intervals running simultaneously
4	User leaves and returns 10 times	10 intervals running — high memory usage

Step	Result	Memory State
5	After several minutes	Browser memory full — tab crashes

4.4 Why This is Dangerous

Real scenario — WebSocket connection without cleanup:

```
useEffect(() => {
  const ws = new WebSocket("wss://api.example.com");

  ws.onmessage = (event) => {
    console.log("Message received:", event.data);
  };
  // No ws.close() cleanup
}, []);
```

Each time the component mounts, a NEW WebSocket connection opens. Old connections stay open forever. The browser eventually runs out of connections and the application breaks.

5. Memory Leak Case Study

5.1 Scenario: Modal Component Without Cleanup

```
function BadModal({ isOpen, onClose }) {
  useEffect(() => {
    if (isOpen) {
      document.body.style.overflow = "hidden";
    }
    // No cleanup — body remains scroll-locked forever
  }, [isOpen]);

  useEffect(() => {
    const handleEscape = (e) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", handleEscape);
    // No cleanup — event listener keeps growing
  }, [onClose]);

  if (!isOpen) return null;
  return <div className="modal">Modal Content</div>;
}
```

5.2 Consequences

Problem	Result
Body scroll never restored	User cannot scroll page after modal closes
Multiple event listeners accumulate	Each modal mount adds a NEW listener
Memory usage increases	After 50 modal opens, 50 listeners are running

6. The Solution: Proper Cleanup Pattern

6.1 Correct Code Pattern

```
function GoodModal({ isOpen, onClose }) {  
  // Cleanup for body scroll  
  useEffect(() => {  
    if (isOpen) {  
      document.body.style.overflow = "hidden";  
    }  
    // CLEANUP: Restore body scroll  
    return () => {  
      document.body.style.overflow = "unset";  
    };  
  }, [isOpen]);  
  
  // Cleanup for event listener  
  useEffect(() => {  
    const handleEscape = (e) => {  
      if (e.key === "Escape") onClose();  
    };  
    window.addEventListener("keydown", handleEscape);  
    // CLEANUP: Remove event listener  
    return () => {  
      window.removeEventListener("keydown", handleEscape);  
    };  
  }, [onClose]);  
  
  if (!isOpen) return null;  
  return <div className="modal">Modal Content</div>;  
}
```

6.2 Common Cleanup Patterns

Scenario	Setup Code	Cleanup Code
Interval	const id = setInterval(fn, ms)	clearInterval(id)
Timeout	const id = setTimeout(fn, ms)	clearTimeout(id)
Event Listener	window.addEventListener(event, fn)	window.removeEventListener(event, fn)

Scenario	Setup Code	Cleanup Code
WebSocket	<code>const ws = new WebSocket(url)</code>	<code>ws.close()</code>
Subscription	<code>const sub = store.subscribe(fn)</code>	<code>sub.unsubscribe()</code>
Animation Frame	<code>const id = requestAnimationFrame(fn)</code>	<code>cancelAnimationFrame(id)</code>
Body Scroll	<code>document.body.style.overflow = "hidden"</code>	<code>document.body.style.overflow = "unset"</code>

7. Performance Optimization Best Practices

7.1 Quick Reference Table

Practice	Syntax	Benefit
useCallback	<code>const fn = useCallback(() => {...}, [deps])</code>	Prevents function recreation on every render
useMemo	<code>const value = useMemo(() => compute(), [deps])</code>	Caches expensive calculations
React.memo	<code>export default React.memo(Component)</code>	Prevents re-renders if props unchanged
Lazy Loading	<code>const Comp = dynamic(() => import("./Comp"))</code>	Reduces initial bundle size
Proper Keys	<code><li key={item.id}></code>	Helps React identify list items efficiently

7.2 Detailed Examples

useCallback

Without useCallback — function recreated every render:

```
const handleClick = () => {  
  console.log("Clicked");  
};
```

With useCallback — function cached:

```
const handleClick = useCallback(() => {  
  console.log("Clicked");  
});
```

```
}, []);
```

React.memo

```
export default React.memo(Button);
```

Lazy Loading in Next.js

```
import dynamic from "next/dynamic";

const HeavyComponent = dynamic(() => import("./HeavyComponent"), {
  loading: () => <p>Loading...</p>
});
```

8. Code Examples from This Project

8.1 Button Component

Location: `src/components/atoms/Button.tsx`

```
interface ButtonProps {
  label: string;
  variant?: "primary" | "secondary" | "danger" | "outline";
  size?: "sm" | "md" | "lg";
  disabled?: boolean;
}

export default function Button({ label, variant = "primary",
  size = "md", disabled = false }: ButtonProps) {
  const variants = {
    primary: "bg-blue-600 text-white hover:bg-blue-700",
    secondary: "bg-gray-600 text-white hover:bg-gray-700",
    danger: "bg-red-600 text-white hover:bg-red-700",
    outline: "border-2 border-blue-600 text-blue-600 hover:bg-blue-50",
  };
  // ... size and return logic
}
```

8.2 Card Component

Location: `src/components/atoms/Card.tsx`

```
interface CardProps {
  children: React.ReactNode;
  title?: string;
  hover?: boolean;
}
```



```
export default function Card({ children, title, hover = false }: CardProps) {
  return (
    <div className={`bg-white rounded-xl shadow-md p-6 border
      ${hover ? "hover:shadow-xl hover:-translate-y-1 cursor-pointer transition" :
    ""}`}>
      {title} && <h3>{title}</h3>
      <div>{children}</div>
    </div>
  );
}
```

8.3 Modal Component with Cleanup

Location: src/components/molecules/Modal.tsx

```
export default function Modal({ isOpen, onClose, title, children }) {
  useEffect(() => {
    if (isOpen) { document.body.style.overflow = "hidden"; }
    return () => { document.body.style.overflow = "unset"; };
  }, [isOpen]);

  useEffect(() => {
    const handleEscape = (e: KeyboardEvent) => {
      if (e.key === "Escape" && isOpen) { onClose(); }
    };
    window.addEventListener("keydown", handleEscape);
    return () => { window.removeEventListener("keydown", handleEscape); };
  }, [isOpen, onClose]);

  if (!isOpen) return null;
  return <div className="fixed inset-0 z-50 ...">...</div>;
}
```

9. Conclusion

9.1 Key Takeaways

- Technical debt must be addressed early. Using TypeScript, reusable components, and consistent styling prevents future headaches.
- React component lifecycle has three phases: Mounting, Updating, and Unmounting. Understanding these is crucial for proper hook usage.
- Always add cleanup in useEffect when creating intervals, timeouts, event listeners, WebSocket connections, subscriptions, or DOM mutations like body scroll locking.
- Without cleanup, every component mount adds new resources that never get freed, leading to memory leaks and browser crashes.
- Performance optimization tools like useCallback, useMemo, React.memo, and lazy loading help keep applications fast.

9.2 Final Checklist

Item	Status
All components have TypeScript interfaces	YES
Zero custom CSS files (Tailwind only)	YES
Modal includes proper useEffect cleanup	YES
Atomic Design principles followed	YES
Responsive layout with Tailwind grid	YES
Research report documents technical debt and performance optimization	YES

10. References

React Official Documentation — useEffect

<https://react.dev/reference/react/useEffect>

React Official Documentation — Managing Memory

<https://react.dev/learn/you-might-not-need-an-effect>

Next.js Documentation — Optimizing Performance

<https://nextjs.org/docs/app/building-your-application/optimizing>

Tailwind CSS Documentation

<https://tailwindcss.com/docs>

TypeScript Handbook

<https://www.typescriptlang.org/docs/>

Atomic Design by Brad Frost

<https://atomicdesign.bradfrost.com/>

Report Prepared By: Tayyaba Shakoor
Date: 03 June 2026 | Internship: Prosensia — Full Stack Development